

Diseño y Configuración de Infraestructura de Dominio de una Empresa

En esta ocasión yo me he encargado de la creación de los objetos del dominio para lo cual he creado un script con las siguientes partes.

Parámetros

En la sección de parámetros he ido añadiendo los parámetros de entrada del script.

```
param(
    [Parameter(Mandatory = $true)]
    [ValidateScript({ Test-Path $_ -PathType Leaf })]
    [string]$CsvPath,

    [string]$Domain = "alicia.local",
    [string]$LogFile
)
```

Fichero log

En esta parte se he creado una variable cuyos datos se añadirán a un fichero y se utiliza una función que se encarga de escribir el mensaje del log.

A lo largo de todo el script se encuentran pequeñas líneas de comando que van añadiendo al fichero información sobre los objetos y su creación.

```
# Resolver ruta del script y del log
$rutaScript = Split-Path -Parent $MyInvocation.MyCommand.Path
if (-not $LogFile) {
    $LogFile = Join-Path $rutaScript "CrearAD_Grupo6_Log.txt"
}

# Preparar los log
if (-not (Test-Path (Split-Path $LogFile))) {
    New-Item -ItemType Directory -Path (Split-Path $LogFile) -Force | Out-Null
}
"" | Out-File -FilePath $LogFile -Encoding UTF8
```

Módulo Active Directory

La función de esta sección es cargar el módulo Active Directory para permitir el uso de algunos cmdlets concretos como puede ser Get-ADUser

```
# Importar modulo de Active Directory
try {
    Import-Module ActiveDirectory -ErrorAction Stop
    Write-Log "Modulo ActiveDirectory cargado correctamente"
}
catch {
    Write-Error "No se pudo cargar el modulo ActiveDirectory"
    exit 1
}
```

CSV

Para la gestión del CSV y que se creen los objetos con la información que se encuentra en el archivo utilizo un bloque para importar el archivo y posteriormente se sacará la información de los nombres de los objetos de ese mismo CSV.

```
# Importar el archivo CSV
$usuarios = Import-Csv -Path $CsvPath
Write-Log "CSV cargado correctamente desde $CsvPath"
```

Unidades organizativas (UO)

En esta sección primero defino las unidades organizativas utilizando un array y posteriormente se creo las unidades organizativas asegurándose con anterioridad de que no existan.

```
$nombresOU = @(
    "OUGrupo6_Usuarios",
    "OUGrupo6_Equipos",
    "OUGrupo6_Administracion"
)
```

```
# Creacion de las UOs
foreach ($ou in $nombresOU) {
    Ensure-OU -NombreOU $ou
}
```

Grupos

En esta sección se utilizó un array para definir los grupos que se deseó crear y posteriormente se les crea en caso de que no existan ya. Los grupos serán de seguridad global.

```
$nombresGrupos = @(
    "GGGrupo6_Admins",
    "GGGrupo6_Empleados"
)
```

```
# Creacion de los grupos de seguridad
foreach ($grupo in $nombresGrupos) {
    $grupoExiste = Get-ADGroup -Filter "Name -eq '$grupo'" -ErrorAction SilentlyContinue

    if (-not $grupoExiste) {
        try {
            $rutaGrupo = "OU=OUGrupo6_Usuarios,$dnDominio"
            New-ADGroup -Name $grupo -GroupScope Global -GroupCategory Security -Path $rutaGrupo
            Write-Log "Grupo creado: $grupo"
        }
        catch {
            Write-Log "ERROR creando grupo $grupo : $($_.Exception.Message)"
        }
    }
    else {
        Write-Log "Grupo ya existe: $grupo"
    }
}
```

Usuarios y asignarlos a los grupos

Para crear los usuarios el bloque que he utilizado comprueba que estos no existan, después crea los usuarios y los habilita para que estén listos para ser usados.

Después en el siguiente bloque he añadido los usuarios al grupo al que deban pertenecer utilizando el archivo CSV como fuente de información.

```
# Creacion de los usuarios
foreach ($usuario in $usuarios) {
    $nombre = $usuario.Nombre.Trim()
    $apellidos = $usuario.Apellidos.Trim()
    $usuarioAD = $usuario.Usuario.Trim()
    $nombreOU = $usuario.OU.Trim()
    $gruposCelda = $usuario.Grupos
    $dnOU = "OU=$nombreOU,$dnDominio"
    $nombreMostrado = "$nombre $apellidos"
    $usuarioExiste = Get-ADUser -Filter "SamAccountName -eq '$usuarioAD'" -ErrorAction SilentlyContinue
    if (-not $usuarioExiste) {
        try {
            $contrasenaSegura = ConvertTo-SecureString $usuario.Contrasena -AsPlainText -Force
            New-ADUser `
                -Name $nombreMostrado `
                -GivenName $nombre `
                -Surname $apellidos `
                -SamAccountName $usuarioAD `
                -UserPrincipalName "$usuarioAD@$Domain" `
                -AccountPassword $contrasenaSegura `
                -Path $dnOU `
                -Enabled $true `
                -ChangePasswordAtLogon $true
            Write-Log "Usuario creado: $usuarioAD"
        }
        catch {
            Write-Log "ERROR creando usuario $usuarioAD : $($_.Exception.Message)"
            continue
        }
    }
    else {
        Write-Log "Usuario ya existe: $usuarioAD"
    }
    # Asignar los grupos a los usuarios
    if ($gruposCelda) {
        $listaGrupos = $gruposCelda -split ';' | ForEach-Object { $_.Trim() }
        foreach ($grupoAsignar in $listaGrupos) {
            try {
                Add-ADGroupMember -Identity $grupoAsignar -Members $usuarioAD -ErrorAction Stop
                Write-Log "Añadido $usuarioAD al grupo $grupoAsignar"
            }
            catch {
                Write-Log "ERROR añadiendo $usuarioAD al grupo $grupoAsignar : $($_.Exception.Message)"
            }
        }
    }
}
}
```

Parte final

Al final del script he utilizado un par de líneas para comprobar que el script ha funcionado correctamente. Una de ellas añade al archivo de logs una línea de texto donde indica que el script ha finalizado correctamente y la otra lo que hace al terminar la creación de los objetos correctamente es mostrar por terminal un texto que le indique a quien ejecute el archivo que ha funcionado correctamente.

Aclaraciones

Es posible que el archivo CSV no se vea correctamente en Excel ya que Excel no interpreta correctamente las separaciones por lo que habrá que ir a datos dentro del Excel y elegir la opción De texto/CSV para que se vea correctamente.

En las imágenes mostradas no está correctamente puesto el nombre del dominio, pero eso lo he corregido para la implementación final.